

Relazione Progetto : Space Java

Programmazione Orientata agli Oggetti

Andrea Ricci Lukas Magalotti Alessandro Tomè Enes Sela

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	4
2	Design	5
2.1	Architettura	5
2.2	Design Dettagliato	7
2.2.1	Andrea Ricci	7
2.2.2	Lukas Magalotti	11
2.2.3	Alessandro Tomè	15
2.2.4	Enes Sela	19
3	Sviluppo	23
3.1	Testing automatizzato	23
3.2	Note di sviluppo	24
3.2.1	Andrea Ricci	24
3.2.2	Lukas Magalotti	24
3.2.3	Alessandro Tomè	25
3.2.4	Enes Sela	26
4	Commenti Finali	28
4.1	Autovalutazioni e lavori futuri	28
4.1.1	Andrea Ricci	28
4.1.2	Lukas Magalotti	28
4.1.3	Alessandro Tomè	29
4.1.4	Enes Sela	30
A	Guida Utente	31
A.1	Menu Principale e Navigazione	31
A.2	Il Negozio (Selezione Skin)	32
A.3	Gameplay e Controlli di Gioco	33
A.4	Potenziamenti e Progressione	34
B	Esercitazioni di laboratorio	35
B.1	Andrea Ricci (andrea.ricci60@studio.unibo.it)	35
B.2	Alessandro Tomè (alessandro.tome@studio.unibo.it)	35
B.3	Enes Sela (enes.sela@studio.unibo.it)	36

Capitolo 1

Analisi

Il gruppo si pone come obiettivo quello di realizzare un gioco chiamato "Space Java" ispirato al famoso gioco arcade *Space Invaders*, con l'aggiunta di ulteriori meccaniche di gioco come abilità, potenziamenti e un progressivo aumento di difficoltà e varietà dei nemici. Abbiamo voluto riprendere il classico gioco di *Space Invaders* aggiungendo una sfumatura "roguelike" (genere di videogiochi caratterizzati da generazione procedurale di nemici, casualità nel gameplay e morte permanente) e un sistema di abilità ispirato al gioco "Vampire Survivors". Il gioco è formato da tre tipi di ondate che si ripetono per ciascun livello a difficoltà incrementale (facile, intermedia, boss), dove sconfitta ogni ondata il giocatore può scegliere una abilità tra tre casuali proposte.

1.1 Descrizione e requisiti

Il software, nato dalla passione del gruppo per il retrogaming, ha come obiettivo la realizzazione di un videogioco d'azione a tema spaziale denominato "Space Java", fortemente ispirato al celebre titolo arcade *Space Invaders*. L'utente piloterà una navicella spaziale, in grado di muoversi esclusivamente lungo l'asse orizzontale, con lo scopo di respingere e sconfiggere continue ondate di nemici ostili.

Rispetto alla formula classica, l'applicazione introduce una marcata sfumatura "roguelike". Per videogioco *roguelike* si intende un titolo caratterizzato dalla generazione procedurale delle sfide, da una forte casualità nel gameplay e dalla "morte permanente", ovvero l'azzeramento dei progressi e l'obbligo di ricominciare la partita dall'inizio in caso di distruzione della navicella.

A questo si aggiunge un sistema di progressione ispirato al gioco *Vampire Survivors*. Il flusso di gioco è strutturato in cicli di tre ondate a difficoltà incrementale (facile, intermedia e "boss"). Al termine di ciascuna ondata, il sistema proporrà all'utente tre potenziamenti o abilità generati casualmente; l'utente dovrà sceglierne uno per migliorare le caratteristiche della propria navicella (es. maggiorazione dei punti vita, aumento dei danni, incremento della velocità o armi differenti). Questi miglioramenti sono fondamentali per permettere al giocatore di sopravvivere, poiché con il passare del tempo la difficoltà del gioco, la resistenza e la forza dei nemici aumenteranno progressivamente.

Requisiti funzionali

- **Controllo utente** = il sistema deve permettere all'utente di pilotare la navicella lungo l'asse orizzontale e di gestire l'emissione dei proiettili.
- **Gestione nemici e ondate** = il software deve generare e gestire il movimento e gli attacchi di diverse tipologie di nemici, organizzandoli in ondate cicliche di difficoltà crescente.
- **Sistema di abilità e potenziamenti** = al completamento di ogni ondata, il sistema deve mettere in pausa l'azione, generare tre potenziamenti casuali (es. vita extra, velocità, armi avanzate) e applicare la scelta dell'utente alla navicella.
- **Gestione dello stato di gioco** = il software deve fornire un'interfaccia con menù di avvio, tracciare il punteggio in tempo reale, salvare le statistiche (come il record di punteggio massimo) e gestire la schermata di fine partita con relativa opzione di riavvio.
- **Personalizzazione estetica (Opzionale)** = il sistema permetterà all'utente di modificare l'aspetto visivo della propria navicella.
- **Modalità infinita (Opzionale)** = il sistema prevederà un ciclo di gioco potenzialmente infinito, continuando a generare ondate fino alla sconfitta inevitabile dell'utente.

Requisiti non funzionali

- **Fluidità e Reattività** = il gioco deve garantire un'elevata responsività agli input dell'utente e gestire simultaneamente il movimento e le collisioni di decine di entità (navicella, nemici, proiettili) senza alcun rallentamento percettibile, per mantenere l'esperienza immersiva.
- **Usabilità dell'interfaccia** = i menù, gli indicatori a schermo (punteggio, vita rimanente) e le schermate di selezione delle abilità devono risultare chiari, intuitivi e non ostacolare la visibilità dell'area di gioco.
- **Bilanciamento** = il generatore casuale delle abilità e l'incremento progressivo della forza dei nemici devono essere calibrati per offrire una sfida equa, che risulti stimolante ma mai impossibile a causa della sola sfortuna.

1.2 Modello del Dominio

Il dominio applicativo di “Space Java” ruota attorno alla gestione di una sessione di gioco spaziale in cui un **Giocatore** (*Player*) affronta ondate crescenti di **Nemici** (*Enemy*). Il sistema è centralizzato da un gestore principale del gioco (*GameManager*) che si occupa di smistare l’utente tra la componente giocata e i menu di interazione, tra cui il menu principale e il negozio (*Shop*). Il giocatore controlla una navicella spaziale personalizzabile tramite una *Skin*. Le navicelle possono sparare proiettili (*Projectile*) per abbattere i nemici, i quali a loro volta si dividono in diverse tipologie e possono rispondere al fuoco. Sconfiggendo i nemici, il giocatore accumula un **Punteggio** (*Score*), che funge anche da valuta all’interno dello *Shop* per sbloccare nuove *Skin*.

Per garantire la componente *roguelike*, il gioco genera delle ondate (*Wave*) di difficoltà crescente. Al termine di specifiche ondate, al giocatore viene offerta la possibilità di raccogliere dei **Potenziamenti** (*PowerUp*) che alterano le statistiche della navicella o dei proiettili sparati. Infine, l’intera esperienza, sia nei menu che durante il gameplay, è accompagnata da un gestore audio (*SoundManager*) che riproduce musica ed effetti sonori reattivi agli eventi (come collisioni e spari). Gli elementi costitutivi del problema sono sintetizzati in Figura 1.1.

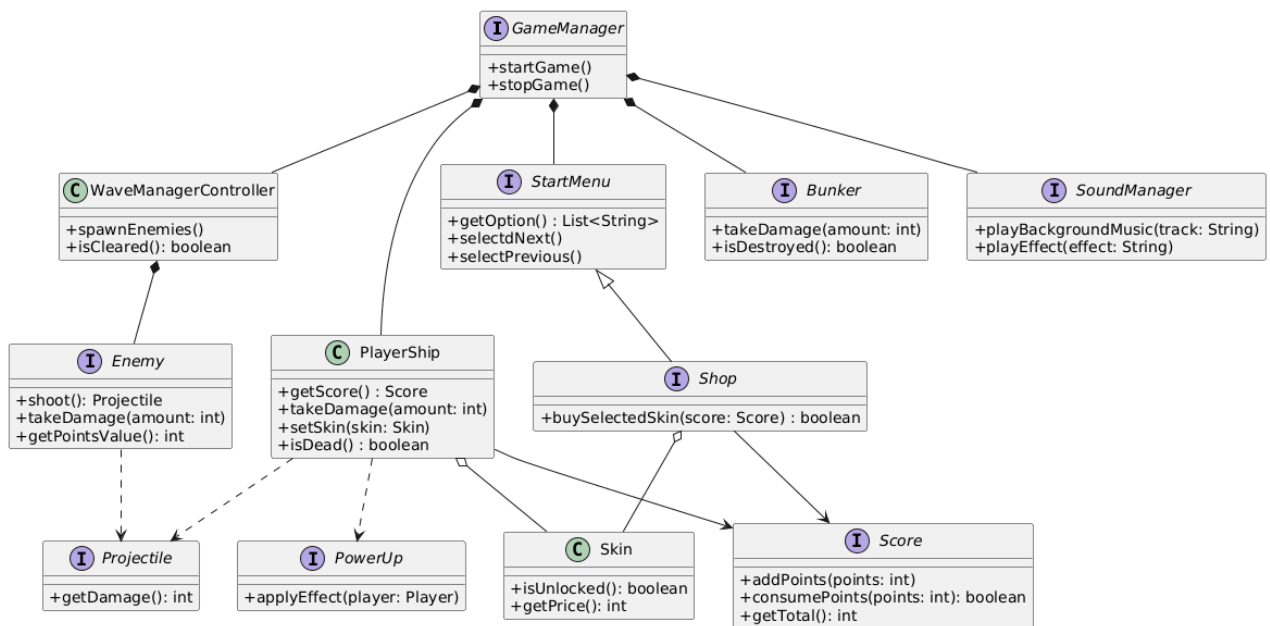


Figura 1.1: Modello del Dominio

Capitolo 2

Design

2.1 Architettura

L'architettura del videogioco "Space Java" è stata progettata seguendo rigorosamente il pattern architetturale MVC (Model-View-Controller). Questa scelta è stata fatta per garantire una separazione netta tra la logica di business (le regole del gioco e lo stato dei menu), la rappresentazione grafica (l'interfaccia utente) e la gestione degli input provenienti dal giocatore. Il sistema si compone di tre macromoduli principali :

- **Model** = rappresenta il nucleo dell'applicazione e detiene l'intero stato del gioco, contiene le entità del dominio (navicelle, nemici, proiettili) e la logica di gestione dei menu (StartMenuModel, Shop). Il Model è completamente agnostico rispetto al livello di presentazione, infatti non importa alcuna libreria grafica e non è a conoscenza di come i dati verranno mostrati a schermo.
- **View** = è il livello di presentazione, implementato avvalendosi della libreria Java Swing. Le classi di questo modulo (come GamePanel, StartMenuView e ShopView) estendono i contenitori standard (JPanel) e hanno l'unico compito di disegnare l'interfaccia. La View non possiede logica decisionale, si limita a leggere lo stato dal Model e a tradurlo visivamente.
- **Controller** = ha la responsabilità di gestire l'interazione dell'utente, è importante notare che i controller del gioco (es. PlayerController, StartMenuController, ShopController) estendono un KeyHandler centralizzato. Intercettano la pressione dei tasti, la interpretano in base al contesto e invocano i metodi appropriati sul Model per modificarne lo stato.

L'intero ciclo di vita di questa triade è orchestrato dalla classe principale di avvio (App) e dal controller principale GameManagerImpl. Quest'ultimo funge da coordinatore centrale ovvero istanzia i componenti MVC, gestisce il CardLayout e scambia le View attive nel pannello principale a seconda dello stato della partita.

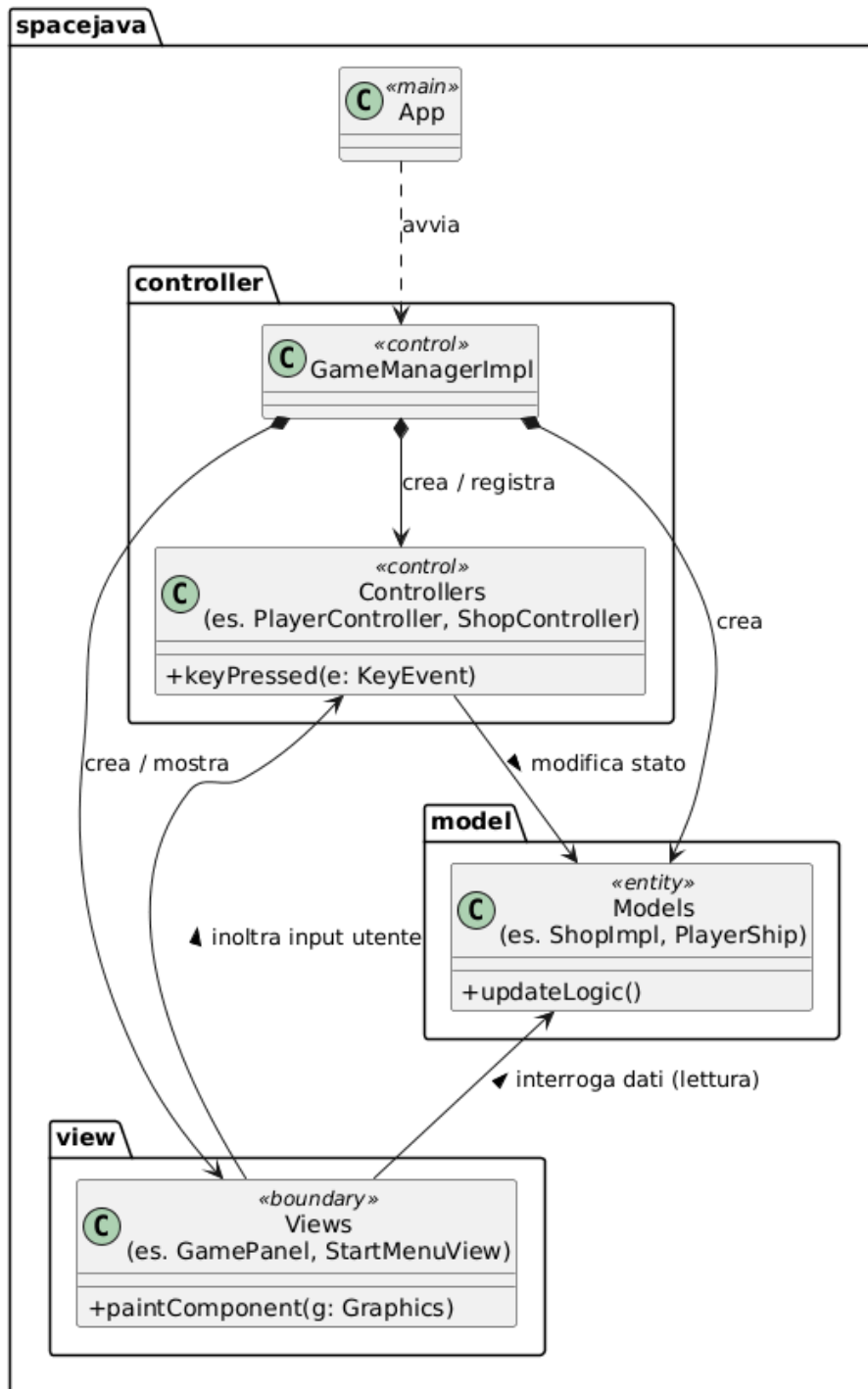


Figura 2.1: Architettura MVC del sistema

2.2 Design Dettagliato

2.2.1 Andrea Ricci

Disaccoppiamento delle Interfacce Grafiche

- **Problema** = le interfacce grafiche dei menu dovevano aggiornarsi in tempo reale al variare delle selezioni dell'utente o dei punti disponibili (es. durante l'acquisto di una navicella). Mantenere un riferimento esplicito della View all'interno del Model avrebbe violato il pattern architetturale MVC, accoppiando la logica di dominio a specifiche librerie grafiche.
- **Soluzione** = si è adottato l'**Observer Pattern**. È stata introdotta l'interfaccia *MenuObserver* che funge da contratto per l'aggiornamento. I Model dei menu (*StartMenuModel* e *ShopImpl*) agiscono da *Observable*: mantengono un observer e gli notificano ad ogni cambiamento di stato (come il cambio di selezione o l'acquisto di un elemento). Le View (*StartMenuView* e *ShopView*) agiscono da *Observer*: implementano l'interfaccia e, alla ricezione della notifica, interrogano autonomamente il Model in sola lettura per ridisegnarsi. Questo garantisce un disaccoppiamento totale, rendendo possibile la sostituzione in blocco dell'interfaccia grafica senza alterare la logica che c'è dietro.

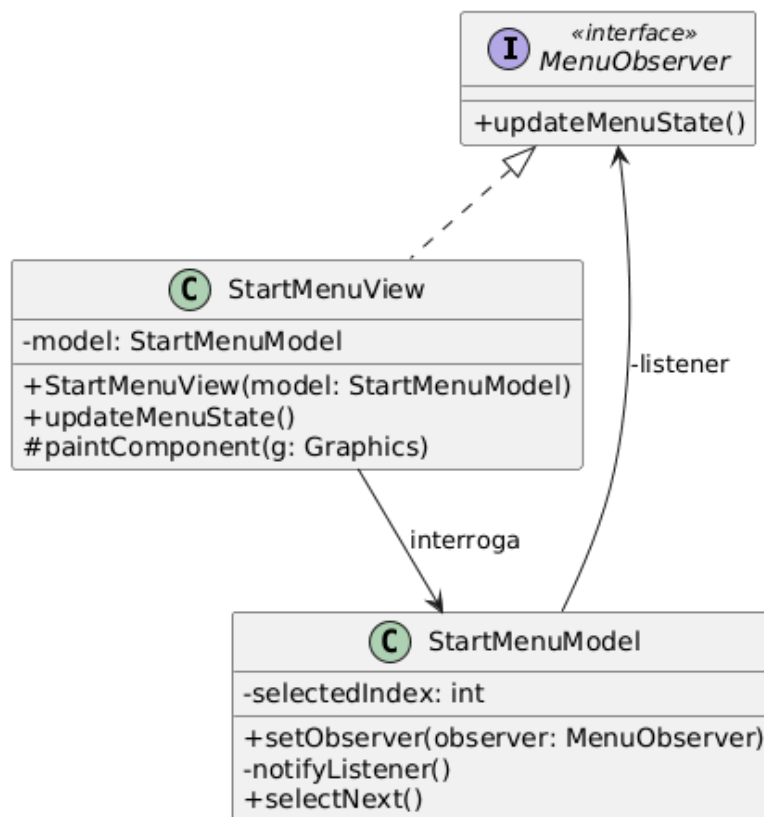


Figura 2.2: Struttura dell'Observer pattern (Menu principale e selezione skin)

Gestione centralizzata dell'Audio

- **Problema** = il sistema audio del gioco richiede l'interazione con le risorse hardware (scheda audio) e l'allocazione di stream in memoria (*Clip*). Consentire la creazione di istanze multiple del gestore sonoro avrebbe potuto causare sovrapposizioni incontrollate della musica di sottofondo e un rapido esaurimento della memoria (memory leak) a causa della mancata chiusura delle risorse allocate per gli effetti sonori frequenti (es. gli spari).
- **Soluzione** = È stato applicato il **Singleton Pattern** alla classe *SoundManagerImpl*, rendendo il costruttore privato e fornendo un unico punto di accesso statico (*getInstance()*), si garantisce che esista una e una sola istanza del motore audio in tutta l'applicazione. L'istanza Singleton gestisce in sicurezza il ciclo di vita delle risorse, chiudendo attivamente gli stream audio al termine della loro esecuzione tramite appositi listener.

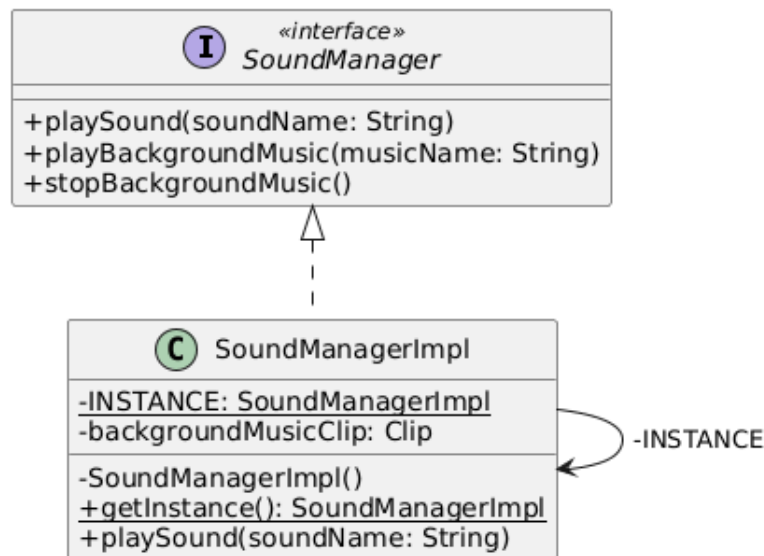


Figura 2.3: Rappresentazione UML della struttura del SoundManager

Navigazione e Azioni dei Menu

- **Problema** = i controller dedicati alla gestione dell'input nei menu (*StartMenuController*, *ShopController*) dovevano poter scatenare eventi ad alto livello, come l'avvio della partita o il cambio di schermata, senza però essere accoppiati direttamente alla classe principale che gestisce il layout delle viste (*GameManagerImpl*).
- **Soluzione** = è stato implementato il **Command Pattern**. Le azioni di navigazione sono state incapsulate in oggetti tramite l'interfaccia funzionale *Command*. I controller dei menu agiscono esclusivamente da *Invoker*: ricevono i comandi nel

costruttore e si limitano a chiamare il metodo *execute()* in risposta all'input dell'utente. Il *Receiver* dell'azione (chi sa effettivamente come cambiare pannello o avviare il livello) rimane confinato al livello architetturale superiore, garantendo controller snelli e altamente riutilizzabili.

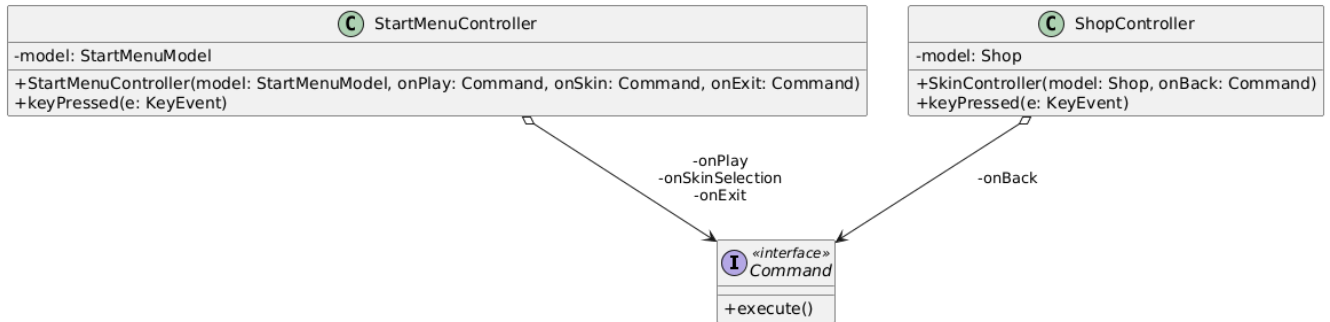


Figura 2.4: Rappresentazione UML del Command Pattern

Gestione e creazioni delle personalizzazioni

- **Problema** = l'inizializzazione dei dati relativi alle personalizzazioni della navicella (le "Skin") presentava un problema di accoppiamento. Scrivere (hardcoding) la lista delle skin disponibili, con i relativi prezzi e percorsi delle immagini, direttamente all'interno dello *ShopImpl* violava il *Single Responsibility Principle (SRP)*. Il Model del negozio si ritrovava a dover gestire non solo la logica transazionale (selezione, acquisto, controllo del saldo), ma anche la fabbricazione dei dati stessi.
- **Soluzione** = per separare la logica dalla creazione dei contenuti, è stato implementato il **Factory Pattern**. È stata creata la classe di utilità *SkinFactory*, che espone il metodo statico *createListOfSkins()*. Lo *ShopImpl* ora si limita a invocare questa factory al momento della sua costruzione, delegando completamente i dettagli su quante e quali skin esistano nel gioco. Questo rende l'aggiunta di future nuove personalizzazioni un'operazione sicura che non intacca in alcun modo la logica di dominio del negozio.

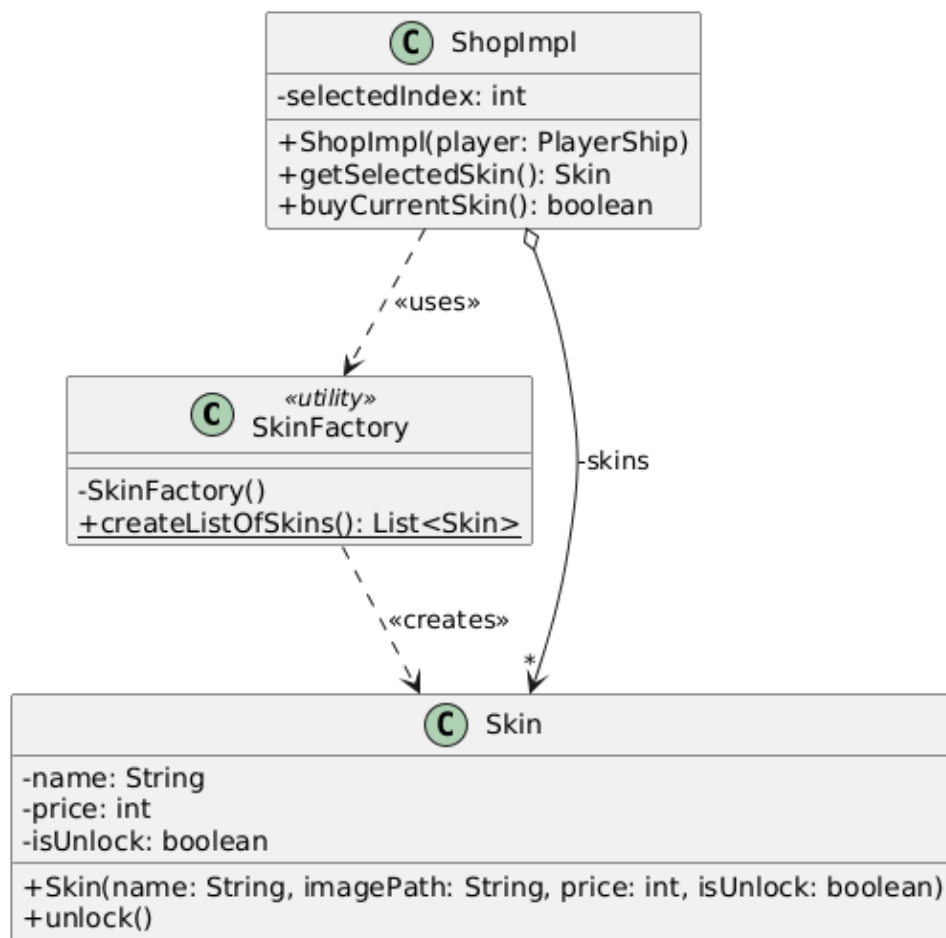


Figura 2.5: Schema UML del Factory Pattern

Gestione e Salvataggio del Punteggio e Transazioni

- **Problema** = in una fase iniziale dello sviluppo, dovevamo decidere in che modo centralizzare lo stato del punteggio. Prima gestita dal *PlayerShip* che non rispetta i vari requisiti della programmazione ad oggetti.
- **Soluzione** = per ripristinare il disaccoppiamento e rispettare il *Single Responsibility Principle* (SRP), la gestione dei punti è stata estratta in un'entità di dominio dedicata tramite l'interfaccia *Score*. La responsabilità di conservare il punteggio è stata assegnata logicamente al giocatore (*PlayerShip*). Il Model del negozio è stato reso puramente transazionale: non detiene alcun riferimento globale al punteggio o al controller, ma riceve l'oggetto *Score* esclusivamente tramite passaggio di parametri al momento dell'acquisto (metodo *buySelectedSkin(Score)*). In questo modo, il controllore torna a svolgere solo il suo ruolo di coordinatore e i Model comunicano in modo pulito e indipendente.
- **Problema** = un altro problema era quello di rendere efficiente il modo in cui viene salvato il *Score* in maniera permanente.

- **Soluzione** = l'ho reso centralizzato il mondo di salvataggio dei punteggi in maniera permanente, rispettando il *Single Responsibility Principle* (SRP). Creando una classe di util con metodi statici *SaveManager*. Che viene utilizzata principalmente dalla classe *ScoreImpl* per poter caricare dal file il punteggio totale e il miglior punteggio una volta finita la partita.

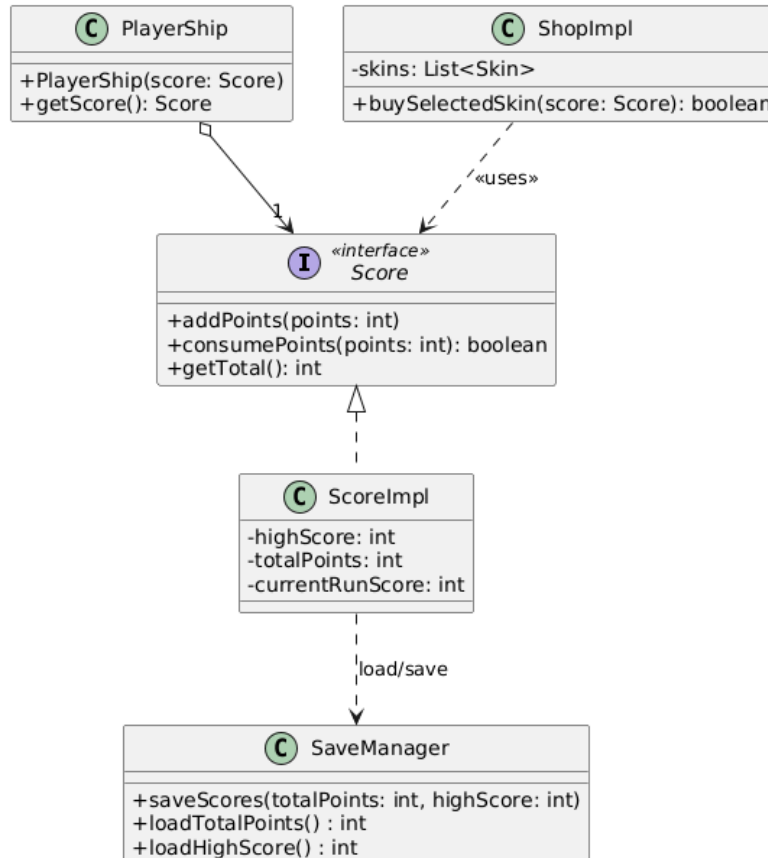


Figura 2.6: Rappresentazione UML dell'implementazione dello Score

2.2.2 Lukas Magalotti

Gestione tipologie di nemici

- **Problema** = per rendere il gameplay vario e interessante, il gioco necessita di diverse tipologie di nemici. Questi devono presentare statistiche uniche, pur condividendo una base comune di comportamenti (movimento, gestione della salute, attacco), rischiando così un'alta duplicazione del codice.
- **Soluzione** = è stata definita un'interfaccia *Enemy* e implementata una **classe astratta** *AbstractEnemy* per centralizzare la logica condivisa. *AbstractEnemy* contiene l'implementazione dei metodi comuni (getter della posizione, gestione del danno subito, controlli sullo stato di morte e logica d'attacco di base). Le classi concrete dei nemici avanzati estendono questa classe base e implementano metodi unici per

gestire il potenziamento delle proprie statistiche all'aumentare della difficoltà e per esporre tali dati all'interfaccia utente (HUD).

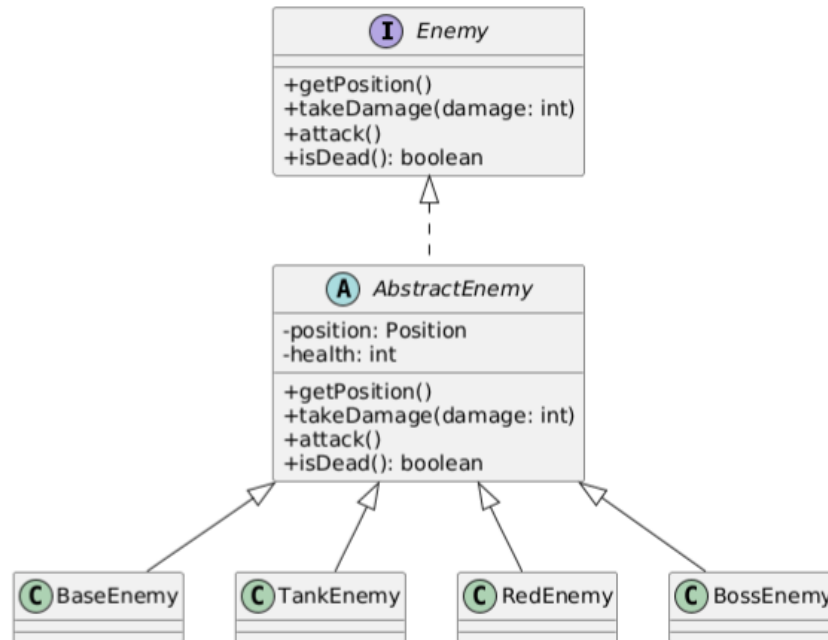


Figura 2.7: Gerarchia delle classi dei nemici

- **Problema** = la tipologia del nemico è un'informazione condivisa e trasversale a diverse classi. Utilizzare stringhe o interi per identificarli potrebbe causare incongruenze e ridotta leggibilità.
- **Soluzione** = è stato introdotto l'**enum** *EnemyType*, che definisce in modo sicuro le tipologie di nemici disponibili (BASE, TANK, RED, BOSS). Questo approccio garantisce la type safety in tutto il progetto e viene sfruttato trasversalmente, ad esempio dal *GamePanel*, per associare e disegnare correttamente gli asset grafici in base al tipo di nemico.

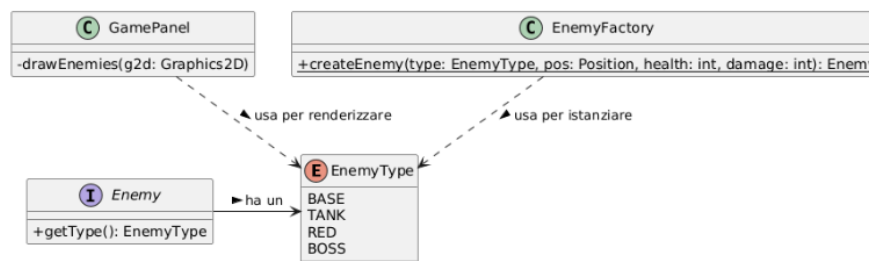


Figura 2.8: Gestione Tipologia Nemico

Creazione dei nemici

- **Problema** = la logica di creazione dei nemici gestita da *WaveManagerController* causa vari problemi:
 1. *Forte Accoppiamento* = il Controller conosce l'esistenza e i dettagli implementativi di ogni singola classe dei nemici.
 2. *Violazione del Principio di Singola Responsabilità* = il Controller ha due compiti: gestire la logica di movimento/attacco dell'ondata e gestire la logica di creazione dei nemici.
 3. *Difficoltà di Manutenzione* = ogni volta che si vuole aggiungere un nuovo tipo di nemico al gioco, è necessario modificare pesantemente il codice del Controller, aggiungendo nuovi *if* o ampliando gli *switch*.
- **Soluzione** = viene usato il pattern **Simple Factory** per delegare la creazione degli oggetti nemici alla classe *EnemyFactory*. In questo modo *WaveManagerController* gestisce solo l'astrazione *enemy* e viene usata una parametrizzazione type safe attraverso l'*enum* *EnemyType*.

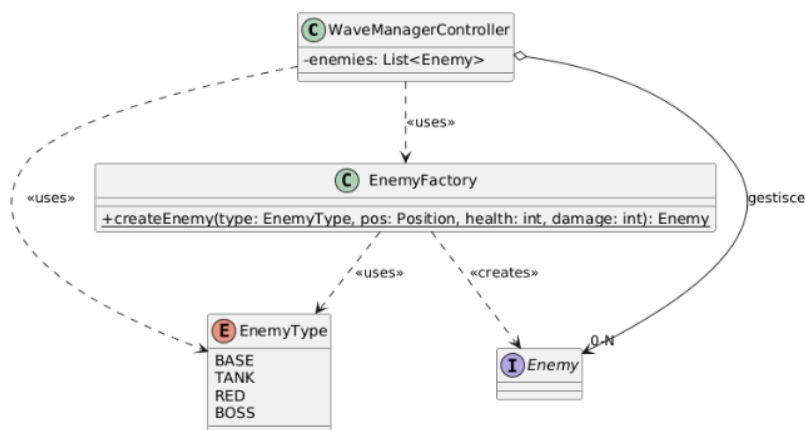


Figura 2.9: Figura per la gestione delle **Wave**

Comportamento dei nemici

- **Problema** = i nemici possono essere complicati da gestire, hanno il proprio stato (salute, posizione), il movimento, le collisioni e il ciclo di vita dei proiettili che sparano, non possono essere gestiti interamente dalla loro classe perché diventerebbe complessa, violando il Principio di Singola Responsabilità.
- **Soluzione** = la responsabilità è stata divisa. Il movimento è gestito per orda dalla classe *WaveManagerController*, l'entità nemico si limita a eseguire l'azione *attack* generando un'istanza di *ProjectileImpl* in una data posizione. Il ciclo di vita di questi proiettili (movimento verso il basso, pulizia quando escono dallo schermo e

controlli di collisione) è invece delegato interamente all'istanza di *EnemyProjectileController* (ricevuta tramite Dependency Injection). Questo mantiene le classi dei nemici semplici e focalizzate solo su loro stesse.

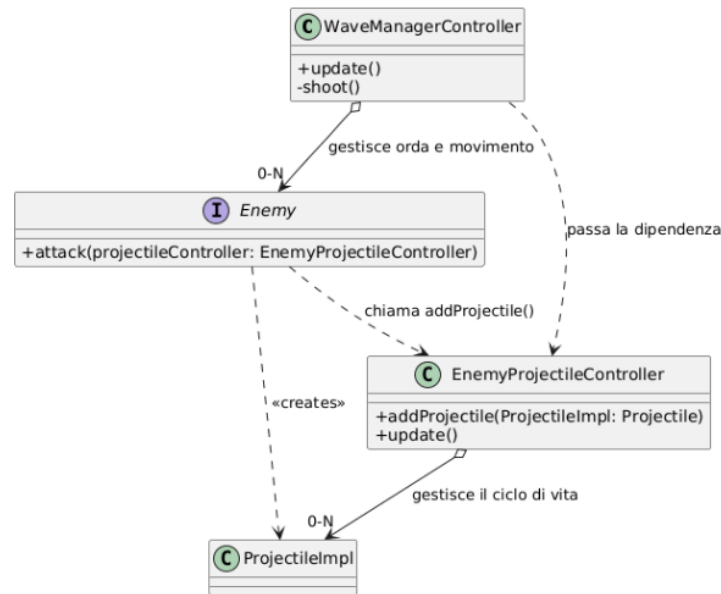


Figura 2.10:

- **Problema** = Come già accennato in precedenza, con l'avanzamento delle ondate alcuni nemici devono diventare più forti per rendere il gioco interessante. Creare nuove classi per ogni livello di difficoltà porterebbe ad avere codice ridondante. D'altro canto, utilizzare variabili statiche all'interno dei Modelli per tracciare il potenziamento avrebbe introdotto un "God Object" e uno stato globale indivisibile, violando il pattern MVC.
- **Soluzione** = La gestione della difficoltà è stata demandata interamente al WaveManagerController, rispettando la separazione delle responsabilità. Il controller tiene traccia dello stato della vita e danno per ogni tipologia di nemico. Ad ogni avanzamento, tramite il metodo `increaseDifficulty()`, il controllore aggiorna queste variabili di istanza e le passa alla EnemyFactory.

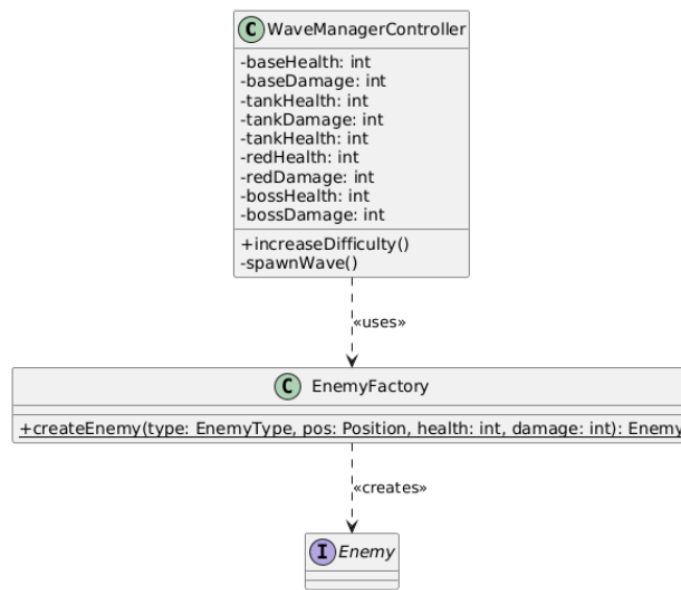


Figura 2.11: Figura che mostra il Factory Pattern dei Nemici

Rimozione dei Magic Numbers e Sicurezza sui Tipi

- **Problema** = L'uso di numeri interi magic numbers per selezionare casualmente i nemici da generare o le statistiche da potenziare rendeva la logica di spawn e upgrade fragile, poco leggibile e prona a errori in fase di espansione.
- **Soluzione** = Sono state introdotte strutture dati basate su enum. È stato creato un array di filtro RANDOM_SPAWN_TYPES contenente tipi specifici dell'enum EnemyType per limitare in modo type-safe quali nemici possono apparire casualmente nella griglia (escludendo il Boss). Similmente, è stata introdotta l'enum UpgradeType all'interno del controllore per determinare in modo esplicito quale statistica incrementare. Questa soluzione ha eliminato l'uso di numeri magici negli switch per la creazione e miglioramento dei nemici.

2.2.3 Alessandro Tomè

Inizializzazione Dinamica del Giocatore

- **Problema** = all'avvio della partita, la navicella del giocatore doveva essere generata tenendo conto di due fattori esterni alla classe stessa: le dimensioni dello schermo (per posizionarla centralmente in basso) e la personalizzazione visiva scelta dall'utente nel negozio (la Skin). Se la classe *PlayerShip* avesse dovuto calcolare da sola le proporzioni dello schermo o interrogare il modello del negozio per ottenere l'immagine corretta, si sarebbe creato un forte accoppiamento con il sistema di rendering e con i menu, violando l'indipendenza del dominio.
- **Soluzione** = la responsabilità della creazione della navicella è stata affidata al controllore principale della partita (*GameManagerImpl*). Durante la procedura di

startGame(), il manager calcola dinamicamente le coordinate di partenza (*startX* e *startY*) in base alle costanti della finestra di gioco. Successivamente, preleva l'oggetto *Skin* attualmente equipaggiato interrogando lo *SkinModel* e inietta tutte queste informazioni nel costruttore di *PlayerShip*. In questo modo, l'entità del giocatore si limita a ricevere un pacchetto di dati pronti all'uso, rimanendo pura e completamente slegata dalle logiche di interfaccia grafica e di gestione dei menu.

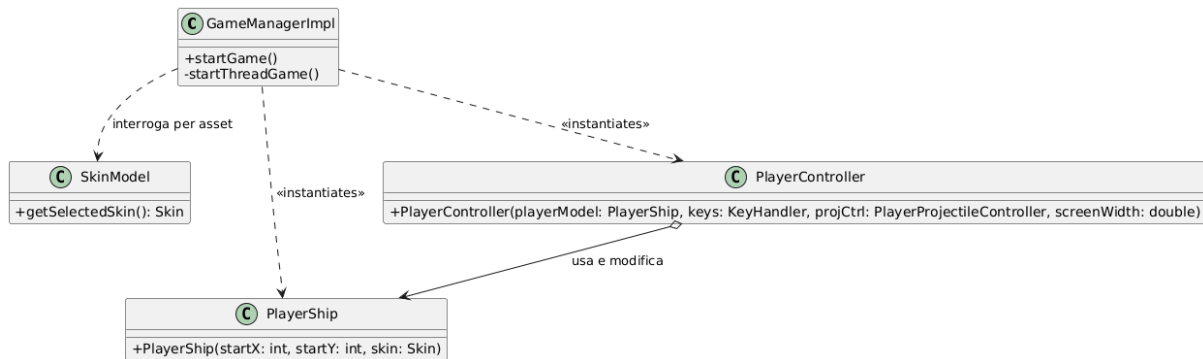


Figura 2.12: Rappresentazione UML del *PlayerShip*

Creazione e Gestione dei Proiettili del Giocatore

- **Problema** = durante il gameplay, l'utente deve poter sparare per difendersi dalle ondate. Se l'entità *PlayerShip* avesse generato da sola i propri proiettili e ne avesse gestito il movimento e i controlli sui limiti dello schermo, la classe sarebbe diventata inutilmente complessa. Inoltre, per permettere ai bunker difensivi di intercettare i colpi amici, sarebbe stato necessario esporre la lista dei proiettili a troppe classi esterne.
- **Soluzione** = la genesi dei proiettili è stata separata dalla loro evoluzione nel tempo. Il *PlayerController* intercetta la pressione del tasto di sparo (Barra Spaziatrice) e gestisce un "cooldown" (tempo di ricarica) basato sui frame al secondo per evitare che l'utente spari un flusso continuo di proiettili. Quando il tempo di ricarica è azzerato, il Controller calcola le coordinate di generazione in base alla posizione attuale della navicella e istanzia un nuovo *ProjectileImpl*. Invece di conservarlo, lo passa immediatamente al *PlayerProjectileController*. Quest'ultimo diventa l'unico proprietario del ciclo di vita del proiettile: ne aggiorna la posizione verso l'alto ad ogni frame e lo rimuove automaticamente dalla memoria non appena le sue coordinate superano il bordo superiore dello schermo, garantendo ottime prestazioni e prevenendo memory leak.

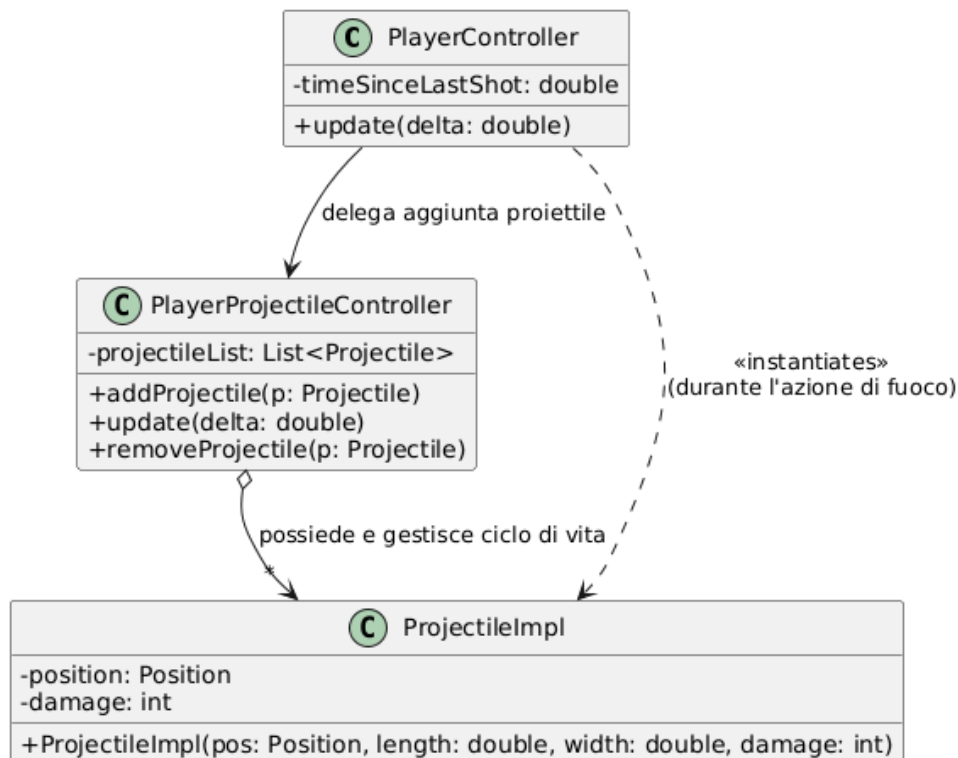


Figura 2.13: Rappresentazione UML dei proiettili del *PlayerShip*

Separazione delle responsabilità dell'entità Giocatore

- Problema** = inizialmente, la gestione dell'entità del giocatore rischiava di concentrare troppe responsabilità all'interno di un'unica classe (*PlayerShip*). Se la navicella avesse dovuto gestire direttamente la lettura degli input da tastiera, il calcolo del cooldown di fuoco, l'istanziamento dei proiettili e il controllo dei limiti dello schermo, si sarebbe verificata una grave violazione del Principio di Singola Responsabilità (*Single Responsibility Principle* - *SRP*). Questo avrebbe reso il Model strettamente accoppiato al sistema di input e all'interfaccia, rendendolo difficilmente manutenibile e impossibile da testare in isolamento.
- Soluzione** = per garantire la massima coesione e il minimo accoppiamento, si è optato per una rigorosa applicazione del pattern architetturale MVC. La classe *PlayerShip* (il Model) è stata ridotta a un puro contenitore di stato e logica fisica di base (coordinate spaziali, punti vita, dimensioni fisiche e Skin equipaggiata). Tutta la logica reattiva è stata demandata a un controllore dedicato, il *PlayerController*. Quest'ultimo riceve in ingresso il *KeyHandler* per interpretare gli input dell'utente, calcola le tempistiche di sparo e aggiorna le coordinate del Model. In questo modo, *PlayerShip* risulta completamente agnostica rispetto alla periferica di input utilizzata (rendendo teoricamente possibile la facile sostituzione della tastiera con un gamepad).

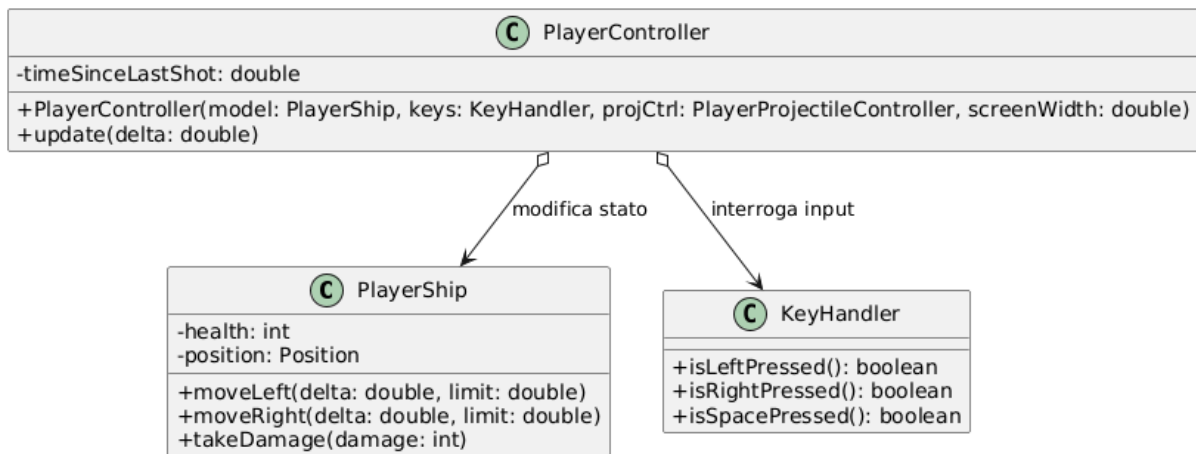


Figura 2.14:

Risoluzione dello Stato Globale tramite Dependency Injection

- Problema :** = durante le fasi di sviluppo, per consentire al giocatore di generare proiettili e per far comunicare i colpi sparati con le entità nemiche e i bunker, l'approccio più immediato risultava essere l'uso di variabili statiche (es. liste globali all'interno di un *PlayerProjectileController* statico, o punteggi salvati staticamente nel giocatore). Tuttavia, l'abuso di metodi e campi statici crea un "God Object" e uno stato globale non controllabile. Questo rompe il principio di incapsulamento dell'OOP, impedisce di eseguire test di unità isolati e preclude del tutto scalabilità future (ad esempio, l'implementazione di una modalità multigiocatore locale in cui due navicelle devono avere liste di sparo indipendenti).
- Soluzione :** = l'architettura del giocatore e delle sue interazioni è stata riprogettata adottando la **Dependency Injection** (Iniezione delle Dipendenze). I controller che gestiscono i proiettili (*PlayerProjectileController*, *EnemyProjectileController*) e l'orchestratore della partita (*GameManager*) sono stati trasformati in istanze concrete prive di stato statico. È il *GameManagerImpl* a istanziarle all'avvio della partita e a "iniettarle" direttamente nei costruttori delle classi che ne necessitano. Ad esempio, il *PlayerController* riceve il *PlayerProjectileController* per aggiungervi nuovi proiettili, mentre il *BunkerController* riceve i controller di ambo le fazioni per calcolarne le collisioni. Questo approccio ha reso il codice modulare e ha permesso la scrittura di Test JUnit efficaci tramite l'utilizzo di *Mock Objects*.

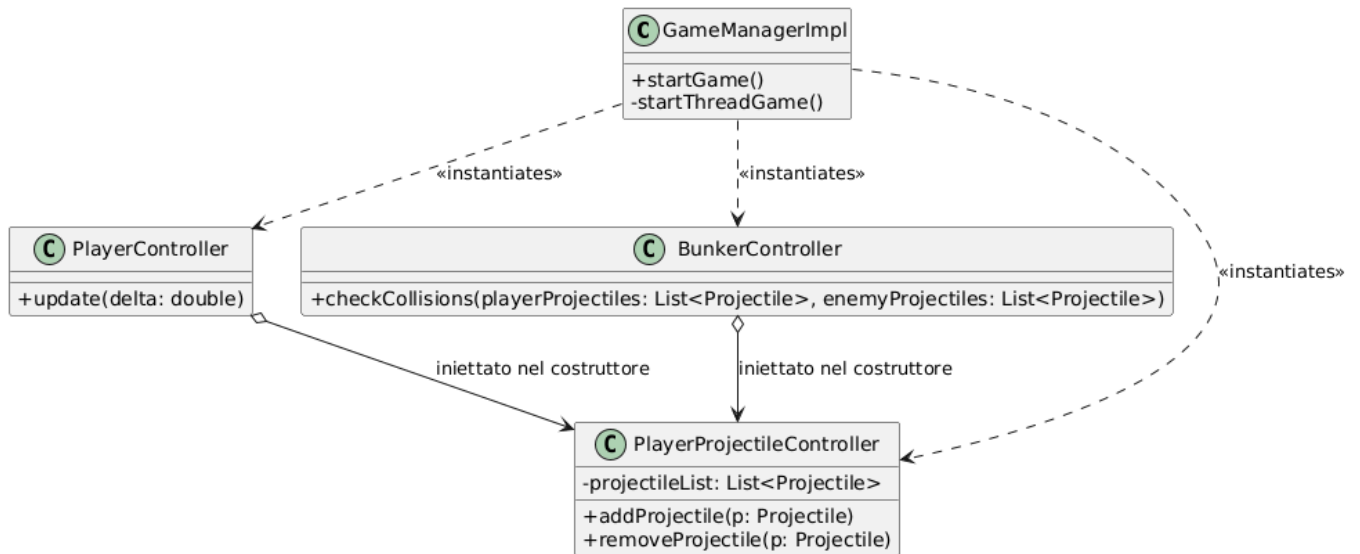


Figura 2.15: Rappresentazione UML del **Dependency Injection**

2.2.4 Enes Sela

Gestione della Schermata di Game Over

- **Problema** = al termine della partita, era necessario gestire una schermata di fine gioco che permettesse all'utente di visualizzare il punteggio ottenuto e di scegliere se riavviare la partita o tornare al menu principale, mantenendo una netta separazione tra la logica di selezione e il rendering grafico.
- **Soluzione** = ho progettato la classe *GameOverModel* seguendo il pattern MVC. Il Model incapsula in totale isolamento lo stato del menu (l'indice dell'opzione selezionata, la lista delle opzioni e il punteggio finale della run). La navigazione è stata resa circolare tramite i metodi *selectNext()* e *selectPrevious()*, in modo che, arrivati all'ultima opzione, la pressione del tasto direzionale riporti automaticamente alla prima. Questo garantisce che la View debba solo interrogare il Model per sapere cosa disegnare, senza contenere alcuna logica condizionale.

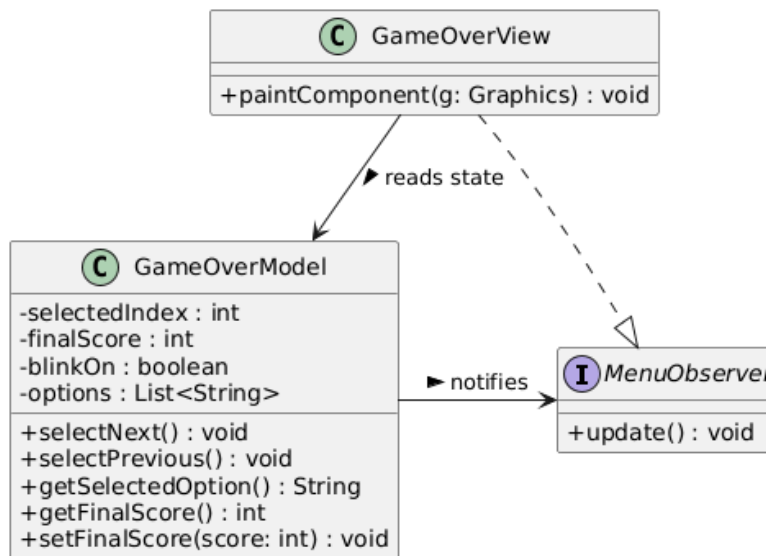


Figura 2.16: Diagramma UML per il sistema di GameOver

Integrazione delle Skin Passive

- **Problema** = le personalizzazioni estetiche (Skin) dovevano non solo cambiare l'aspetto della navicella, ma anche conferire abilità passive (come moltiplicatori di punteggio o scudi speciali). Accoppiare l'entità del giocatore con le logiche dello Shop per leggere questi potenziamenti avrebbe violato l'indipendenza del dominio.
- **Soluzione** = ho esteso la classe *Skin* dotandola di attributi intrinseci di gameplay (es. *scoreMultiplier* e *bossShield*). Sfruttando la *SkinFactory*, i parametri passivi vengono pre-caricati al momento della creazione delle skin disponibili. Quando il giocatore equipaggia una skin, il *PlayerShip* acquisisce automaticamente queste statistiche passive senza dover comunicare con i controller del negozio, mantenendo un'architettura snella e coesa.

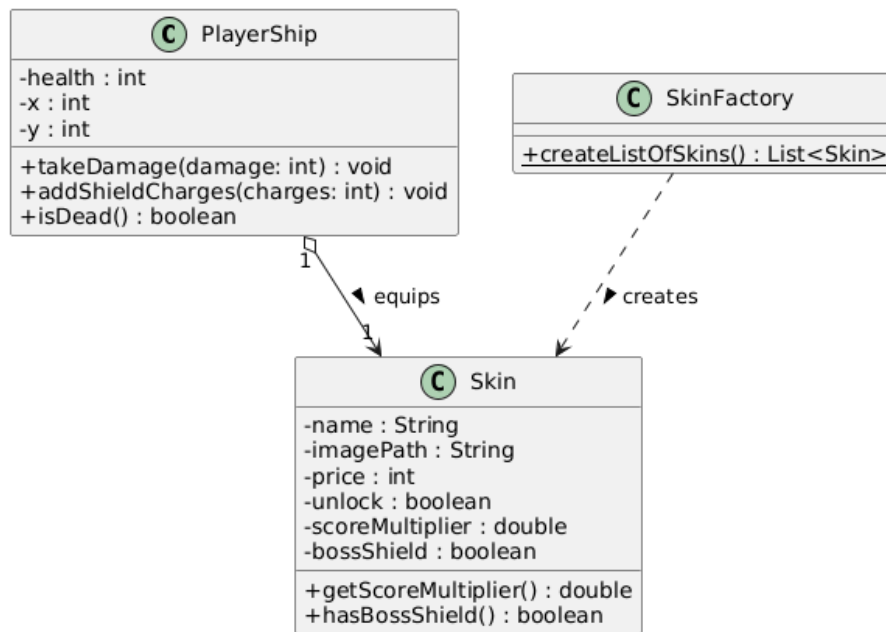


Figura 2.17: Diagramma UML per le Skin Passive e il Giocatore

Sistema di PowerUp

- **Problema** = l'introduzione della meccanica "roguelike" richiedeva l'applicazione di potenziamenti dinamici tra un'ondata e l'altra (es. cura della salute o aggiunta di cariche per lo scudo), alterando lo stato della navicella a runtime.
- **Soluzione** = ho implementato un sistema che, durante la fase di pausa al termine delle ondate, presenta un menu di scelta. L'entità del giocatore espone metodi sicuri (come `addShieldCharges()`) che il sistema di PowerUp invoca una volta confermata la selezione. Questo ha permesso di creare un sistema modulare dove nuovi tipi di PowerUp possono essere aggiunti senza dover modificare la classe centrale della navicella.

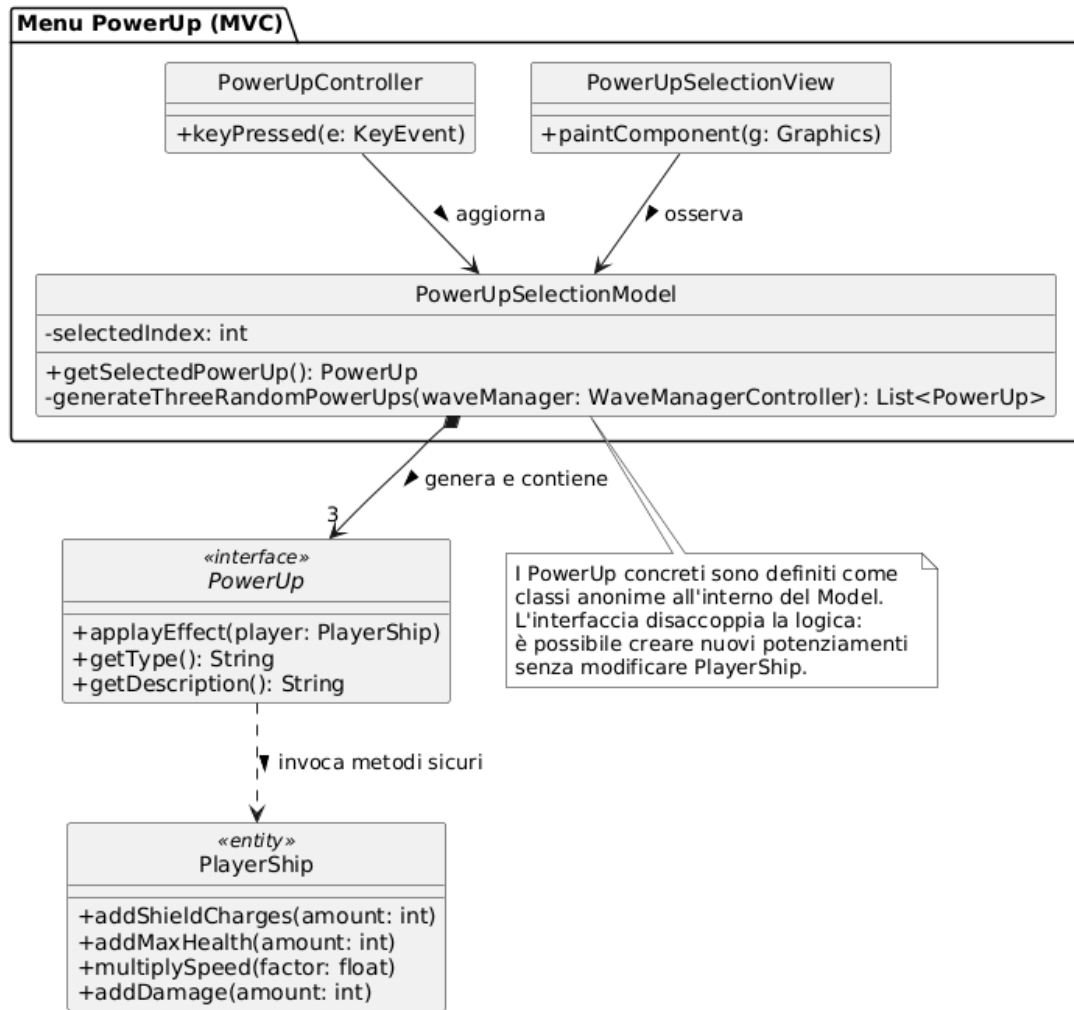


Figura 2.18: Rappresentazione UML del sistema di selezione dei PowerUp

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Come richiesto dagli standard di sviluppo del progetto, per evitare regressioni a fronte degli aggiornamenti, tutte le funzionalità relative al progetto sono state sottoposte a testing completamente automatizzato. Per la stesura e l'esecuzione della test suite è stata impiegata la libreria JUnit 5. Tutti i test implementati si avviano e si concludono in totale autonomia, senza richiedere alcun tipo di intervento o input manuale da parte dell'utente. I componenti e le funzionalità sottoposti a testing automatico comprendono :

- **StartMenu** = è stata verificata la correttezza logica del menu principale, accertando il regolare funzionamento della navigazione tra le diverse opzioni disponibili e valutando l'efficacia del meccanismo di aggiornamento basato sul pattern Observer.
- **Shop** = analogamente a quanto fatto per il menu principale, il testing ha riguardato la navigazione interna e l'integrazione con l'Observer specifico dello shop. Sono state inoltre verificate le logiche transazionali, garantendo che l'acquisto degli elementi avvenga correttamente in tutti gli scenari previsti.
- **Score** = si è proceduto alla validazione del sistema di punteggio, verificando sia l'accuratezza nell'assegnazione dei punti durante le sessioni di gioco, sia la corretta persistenza dei dati sul supporto di memoria permanente.
- **PlayerShip (Modello del giocatore)** = si è testata la coerenza dello stato dell'entità. I test verificano la corretta inizializzazione (coordinate e punti vita previsti), l'applicazione dei danni subiti (incluso il raggiungimento dello stato di morte) e il rispetto dei limiti fisici imposti dallo schermo per impedire movimenti *out-of-bounds*.
- **PlayerProjectileController** = è stata collaudata automaticamente la logica di gestione e il ciclo di vita dei proiettili amici. La suite di test verifica il corretto inserimento e la rimozione dei proiettili dalla lista attiva, il coerente aggiornamento della loro posizione sull'asse verticale al passare dei frame e l'automatica distruzione (rimozione dalla memoria) qualora le coordinate superino il limite superiore dell'area di gioco.
- **Inizializzazione e ciclo di vita dei nemici (EnemyTest)** Verifica la corretta istanziazione delle diverse tipologie di nemici (Base, Tank, Boss, Red) tramite

la factory, controllando i valori iniziali di salute, tipo e coordinate spaziali. Testa l'applicazione dei danni tramite `takeDamage()`, la verifica dello stato di morte del nemico (`isDead()`), la generazione dei proiettili durante l'attacco e le logiche di potenziamento delle statistiche. Quest'ultimo aspetto viene validato accertandosi che la `EnemyFactory` generi correttamente istanze con attributi maggiorati e ispezionando il reale danno dei proiettili istanziati durante la fase di attacco del nemico, garantendo l'integrità del flusso dei dati.

- **Gestione dei proiettili ostili (`EnemyProjectileControllerTest`)** = verifica l'aggiornamento della posizione dei proiettili lungo l'asse Y e valida l'efficacia del meccanismo di rimozione automatica dalla memoria delle entità proiettile non appena queste superano i confini fisici dell'area di gioco (`SCREEN_HEIGHT`).
- **Flusso e movimento delle ondate (`WaveManagerControllerTest`)** = controlla lo stato iniziale del gestore delle ondate, lo spostamento coordinato dei nemici, la corretta eliminazione dei nemici sconfitti all'interno della collezione e il passaggio automatico alla wave successiva.

Tutti i test sopra descritti sono completamente automatici, indipendenti dall'interazione dell'utente e non introducono alcuna forma di testing manuale.

3.2 Note di sviluppo

3.2.1 Andrea Ricci

- *Fuctional Interface* : <https://github.com/eneSSela/00P25-SpaceJava/blob/7ab3a8b36975a2src/main/java/it/unibo/spacejava/api/Command.java#L7>
- *Fuctional Interface* : <https://github.com/eneSSela/00P25-SpaceJava/blob/7ab3a8b36975a267214f6b79dbcc71726f7822a2/src/main/java/it/unibo/spacejava/api/MenuObserver.java#L7>
- *Stream* : <https://github.com/eneSSela/00P25-SpaceJava/blob/7ab3a8b36975a267214f6b79dbcc71726f7822a2/src/main/java/it/unibo/spacejava/controller/BunkerController.java#L71>
- *Utilizzo libreria di controllo audio* : <https://github.com/eneSSela/00P25-SpaceJava/blob/7ab3a8b36975a267214f6b79dbcc71726f7822a2/src/main/java/it/unibo/spacejava/model/sound/SoundManagerImpl.java#L20>

3.2.2 Lukas Magalotti

Di seguito si elencano le funzionalità avanzate del linguaggio e le logiche implementative di maggior rilevanza utilizzate per lo sviluppo dei controller delle ondate di nemici e dei proiettili.

- **Uso di Lambda Expressions e metodi funzionali per le collezioni**

Per la gestione dinamica del ciclo di vita delle entità a schermo, si è fatto ricorso alle API funzionali di Java applicate alle liste. Nello specifico, all'interno dei metodi *update()*, le lambda expression sono state impiegate tramite il metodo *removeIf* per rimuovere in modo sicuro e conciso sia i proiettili che oltrepassano il margine inferiore dello schermo, sia le navicelle nemiche la cui salute è scesa a zero (*isDead()*).

- Permalink: <https://github.com/eneSSela/00P25-SpaceJava/blob/ee6b7e9bb703323390src/main/java/it/unibo/spacejava/controller/WaveManagerController.java#L233-L252>

- Permalink: <https://github.com/eneSSela/00P25-SpaceJava/blob/ee6b7e9bb703323390src/main/java/it/unibo/spacejava/model/enemies/AbstractEnemy.java#L125-L132>

- **Riadattamento di logiche geometriche e algoritmi di movimento**

Per il corretto posizionamento dinamico dei colpi nemici, la logica di origine del proiettile è stata calcolata geometricamente basandosi sulle dimensioni dell'entità, implementando un offset per centrare l'attacco rispetto all'asse X della navicella stessa. Inoltre, l'algoritmo di movimento a sciame dell'ondata è stato implementato monitorando i margini virtuali creati dal nemico più a destra e quello più a sinistra della griglia, invertendo il vettore di marcia e forzando una traslazione sull'asse Y al contatto con i lati dello schermo. Infine il sistema di rilevamento degli impatti tra i proiettili del giocatore e la flotta nemica (*checkhitEnemies()*) non delega il calcolo a librerie esterne, ma utilizza un algoritmo custom di tipo AABB (Axis-Aligned Bounding Box) scritto da Alessandro Tomè. Il sistema valuta l'intersezione pura delle aree nello spazio 2D elaborando dinamicamente le coordinate (X, Y) e le dimensioni fisiche (width, height) delle entità.

- Permalink: <https://github.com/eneSSela/00P25-SpaceJava/blob/ee6b7e9bb703323390src/main/java/it/unibo/spacejava/Utils.java#L45-L54>

- Permalink: <https://github.com/eneSSela/00P25-SpaceJava/blob/ee6b7e9bb703323390src/main/java/it/unibo/spacejava/controller/WaveManagerController.java#L304-L313>

3.2.3 Alessandro Tomè

Nello sviluppo dei componenti legati all'entità del giocatore (*PlayerShip*), alla lettura dei suoi input (*PlayerController*) e alla gestione del ciclo di vita dei suoi proiettili (*Player-ProjectileController*), si è fatto ricorso a diverse feature del linguaggio Java spingendosi oltre i costrutti base. Di seguito ne vengono evidenziate alcune:

- **Uso di Lambda Expressions e Collections API (removeIf)**

Per la pulizia automatica della memoria dai proiettili non più necessari (es. quelli usciti dallo schermo o distrutti), invece di utilizzare i tradizionali e verbosi cicli con

Iterator espliciti, è stata sfruttata l'API funzionale delle collezioni passando una lambda expression al metodo *removeIf*. Questo ha reso il codice molto più leggibile e conciso.

– <https://github.com/eneSSela/00P25-SpaceJava/blob/07acf83b9b1494f3ce06a92441d1src/main/java/it/unibo/spacejava/controller/PlayerProjectileController.java#L49>

- **Uso di Unmodifiable Collections (Programmazione Difensiva)**

Per garantire un rigido incapsulamento (essenziale nell'architettura MVC) ed evitare che classi esterne potessero aggiungere o rimuovere proiettili aggirando i controlli del controller, il metodo *getter* della lista dei proiettili restituisce una collezione immutabile tramite i metodi moderni della libreria standard di Java.

– <https://github.com/eneSSela/00P25-SpaceJava/blob/07acf83b9b1494f3ce06a92441d1src/main/java/it/unibo/spacejava/controller/PlayerProjectileController.java#L31>

- **Iterazione funzionale tramite *forEach***

Per l'aggiornamento della posizione di tutti i proiettili attivi a ogni *tick* del motore fisico, si è optato per l'iterazione funzionale interna. Anziché delegare il controllo a un ciclo *for* esterno, si è passata l'azione da eseguire direttamente al metodo *forEach* della collezione, sfruttando la sintassi funzionale.

– <https://github.com/eneSSela/00P25-SpaceJava/blob/4280308fcee0010dd7b64e8949src/main/java/it/unibo/spacejava/controller/PlayerProjectileController.java#L43>

3.2.4 Enes Sela

Nello sviluppo delle mie funzionalità e in particolare nella stesura dell'estesa suite di test automatizzati (JUnit 5), ho fatto ricorso a diverse API moderne e feature avanzate di Java:

- **Utilizzo intensivo di Stream API e Lambda Expressions**

Per gestire le logiche di collisione o di ricerca all'interno delle liste (ad esempio, per trovare la coordinata Y del nemico più in basso nella *WaveManagerController* o per filtrare i proiettili nei bunker), ho sfruttato l'API Stream di Java, riducendo la complessità ciclomatica e rendendo il codice più dichiarativo. Permalink : <https://github.com/eneSSela/00P25-SpaceJava/blob/ee6b7e9bb70332339019e5552cf9442bfb6f02e0src/main/java/it/unibo/spacejava/controller/WaveManagerController.java#L447>

- **Testing automatizzato**

Ho sviluppato test di unità per verificare la matematica dei poteri passivi delle Skin e

la navigazione circolare della schermata di Game Over. Questi test garantiscono l'assenza di regressioni qualora vengano aggiunte nuove navicelle in futuro. Permalink:

<https://github.com/eneSSela/00P25-SpaceJava/blob/ee6b7e9bb70332339019e5552cf9442b/src/test/java/it/unibo/spacejava/model/SkinPassivesTest.java#L16>

- **Uso della classe Optional**

Nelle operazioni terminali degli Stream (come `max()` per la ricerca della coordinata estrema), ho evitato controlli di nullità manuali facendo uso del costrutto `Optional` e dei suoi metodi sicuri come `orElse()`. Permalink: <https://github.com/eneSSela/00P25-SpaceJava/blob/ee6b7e9bb70332339019e5552cf9442bfb6f02e0/src/main/java/it/unibo/spacejava/controller/WaveManagerController.java#L447>

Capitolo 4

Commenti Finali

4.1 Autovalutazioni e lavori futuri

4.1.1 Andrea Ricci

Ho sempre nutrito un forte interesse per la programmazione object-oriented, da quando alle superiori mi sono cimentato in questo mondo. L'idea di avvicinarmi a un progetto partito da un'idea mia di sviluppare un progetto che si interessa di un hobby che coltivo da quando sono piccolo, i videogiochi. Questo progetto penso che *Space Java*, sia stato un'ottima esercitazione per me e i miei colleghi in modo da poter mettere in pratica gli argomenti affrontati durante il corso in contesti più circoscritti. La prima difficoltà incontrata è stata senza dubbio il lavoro di gruppo: data infatti la mia personalità intraprendente e solitaria ho trovato moderatamente ostico dovermi confrontare con i miei collaboratori prima di adottare una determinata soluzione che mi sembrava potesse funzionare. Tuttavia ritengo che questo fosse proprio uno degli obiettivi principali del corso: imparare, con l'esperienza, ad acquisire maggiore confidenza con il lavoro di squadra, una realtà con cui sarà necessario confrontarsi nel mondo del lavoro.

All'interno del progetto ritengo che il principale punto di forza del mio lavoro risieda nella rigorosa applicazione dell'architettura MVC e dei Design Pattern, cercando di rispettarli come abbiamo visto a lezione. Mi sento dunque soddisfatto del lavoro svolto, pur nonostante ci siano state delle difficoltà e anche situazioni di stallo durante lo sviluppo del progetto, ma nel complessivo mi sono molto divertito durante la progressione del progetto. Provando inoltre anche soddisfazione nel completamento delle mie parti nel miglior modo che ho ritenuto. In futuro sono certo che avrò nuove occasioni per migliorare la qualità del mio codice e imparare altri linguaggi orientati agli oggetti, che mi risulteranno sicuramente più accessibili grazie all'esperienza già maturata con Java.

4.1.2 Lukas Magalotti

Sono da sempre un amante di videogiochi e, in tempi recenti, mi sono avvicinato con molto interesse alle dinamiche roguelike, che abbiamo cercato di portare all'interno di "Space Java". Avere l'opportunità di sviluppare un gioco vero e proprio è stato entusiasmante, ma

anche molto sfidante, infatti mi sono trovato davanti a varie difficoltà. All'inizio, la sfida più grande è stata capire da dove e come iniziare a programmare per trasformare un'idea astratta in un'architettura a oggetti funzionante, che potesse interagire con il codice dei miei compagni. Inoltre, lavorare in un team ha richiesto un grande sforzo di coordinazione; accordarsi con i compagni sulle scelte architetturali e su come far comunicare le nostre rispettive parti di codice è stato fondamentale.

Nonostante queste difficoltà, sono molto soddisfatto del mio lavoro. Ritengo che il punto di forza principale della mia parte (la gestione dei nemici) sia la sua alta estensibilità. Aver centralizzato i comportamenti comuni in `AbstractEnemy` e aver delegato la creazione delle entità a un'apposita `EnemyFactory` ha permesso semplificare il `WaveManagerController`, rispettando il Principio di Singola Responsabilità. Trovo inoltre che l'aver separato il ciclo di vita dei proiettili (`EnemyProjectileController`) e l'aver implementato un potenziamento dinamico privo di stati globali o metodi statici, delegando l'avanzamento della difficoltà al Controller e l'assemblaggio logico alla Factory, abbia reso il sistema architetturalmente puro e perfettamente in linea con le rigide regole dell'Object-Oriented Programming, rendendolo nettamente superiore alla creazione di decine di classi ridondanti. Ritengo che la realizzazione di questo progetto mi abbia fornito preziose competenze per il mio futuro. Ho compreso in prima persona l'importanza di dedicare il giusto tempo alla fase di progettazione architetturale prima ancora di procedere con la stesura del codice. Inoltre, ho potuto toccare con mano l'utilità pratica dei Design Pattern per mantenere il codice pulito e ordinato, e ho interiorizzato le dinamiche fondamentali per organizzarsi e coordinarsi efficacemente all'interno di un team.

4.1.3 Alessandro Tomè

Fin dall'inizio di questo corso, ho nutrito un forte interesse per la progettazione del software e per le architetture di sistema. Avere l'opportunità di applicare i concetti teorici dell'Object-Oriented Programming alla realizzazione di un videogioco, unendo lo studio a una mia grande passione, ha reso lo sviluppo di "Space Java" un'esperienza estremamente stimolante e formativa.

Questo progetto ha rappresentato un eccellente banco di prova per consolidare quanto appreso a lezione. La principale difficoltà che ho riscontrato durante lo sviluppo è stata il radicale cambio di mentalità necessario per abbandonare gli approcci procedurali e l'uso di stati globali. Inizialmente, infatti, la tentazione di utilizzare variabili statiche e "scorciatoie" strutturali per far comunicare agilmente il giocatore, i proiettili e i nemici era forte. Tuttavia, comprendere i limiti di questo approccio e dover riprogettare il codice mi ha fatto scontrare con ostacoli architetturali non banali, spingendomi a cercare soluzioni più scalabili ed eleganti. Ritengo che il principale punto di forza del mio contributo risieda proprio nel lavoro di *refactoring* e nella rigorosa applicazione del pattern MVC e della *Dependency Injection*. Essere riuscito a disaccoppiare completamente la logica della navicella (*PlayerShip*) dai controlli di input e dalla gestione dei proiettili, eliminando ogni "God Object" o dipendenza statica, mi ha dato grande soddisfazione. Aver visto come queste scelte abbiano reso il codice altamente modulare e facilmente verificabile tramite i test di unità (JUnit) mi ha ripagato dello sforzo iniziale.

Nonostante alcuni momenti di stallo legati alla risoluzione di conflitti architetturali e all'integrazione delle varie parti del sistema, mi ritengo estremamente fiero del risultato finale ottenuto insieme ai miei colleghi. Coordinarsi in un team e mantenere un codice pulito su un repository condiviso è stata un'ulteriore sfida che mi ha preparato alle dinamiche del mondo del lavoro. Guardando al futuro, sono certo che le competenze acquisite in merito alla progettazione del software e ai Design Pattern mi saranno di inestimabile aiuto. Questa esperienza ha gettato delle basi solide che mi permetteranno di affrontare con maggiore consapevolezza e sicurezza lo sviluppo di sistemi software complessi, non solo in Java, ma in qualsiasi altro linguaggio orientato agli oggetti che incontrerò nel mio percorso professionale.

4.1.4 Enes Sela

Lavorare alla realizzazione di “Space Java” è stata una sfida tanto impegnativa quanto gratificante. Interfacciarmi con un progetto software partendo interamente da zero mi ha fatto comprendere l'enorme divario che esiste tra la stesura di piccoli algoritmi didattici e la progettazione di un sistema complesso e interconnesso.

La difficoltà principale che ho dovuto affrontare è stata l'implementazione della logica delle Skin passive e del sistema dei PowerUp senza infrangere il pattern MVC scelto dal team. Inizialmente, la tentazione di inserire la logica di cura o del potenziamento scudi direttamente all'interno delle View o dei Controller dei menu era forte. Tuttavia, studiando a fondo i Design Pattern, sono riuscito a incapsulare correttamente le abilità all'interno dei Model (la classe *Skin*), permettendo al giocatore di ereditarle in modo pulito e sicuro. Come sviluppi futuri per l'applicazione, mi piacerebbe espandere il sistema dei PowerUp introducendo delle “sinergie” (combinazioni di potenziamenti che, se selezionati insieme, sbloccano abilità speciali), e sostituire il salvataggio dei punteggi locale con un database persistente in cloud, introducendo una classifica globale (Leaderboard) per incentivare la competizione tra i giocatori. In conclusione, questa esperienza mi ha fornito basi molto solide sulla programmazione orientata agli oggetti e sull'importanza del lavoro di squadra tramite repository condivisi, competenze che ritengo fondamentali per la mia futura carriera lavorativa.

Appendice A

Guida Utente

A.1 Menu Principale e Navigazione

All'avvio, il giocatore viene accolto dal Menu Principale, accompagnato dalla colonna sonora di background. La navigazione all'interno dei menu è stata progettata per essere utilizzata interamente da tastiera:

- Utilizzare i tasti **Freccia Su** e **Freccia Giù** per scorrere tra le opzioni disponibili ("Gioca", "Seleziona Skin", "Esci").
- Premere **Invio** per confermare la selezione.



Figura A.1: Come appare l'applicazione all'avvio

A.2 Il Negozio (Selezione Skin)

Selezionando l'apposita voce dal menu, si accede al negozio (Shop), dove è possibile visualizzare la propria "Banca Dati" (i punti totali accumulati nelle partite precedenti) e sbloccare nuove personalizzazioni per la navicella.

- Utilizzare i tasti **Freccia Sinistra** e **Freccia Destra** per scorrere la vetrina delle navicelle disponibili.
- L'interfaccia dinamica mostrerà se una navicella è già sbloccata o, in caso contrario, il suo costo.
- Se si possiedono abbastanza punti, premere la **Barra Spaziatrice** per acquistare la navicella visualizzata. Il saldo verrà aggiornato istantaneamente.
- Premere **Invio** in qualsiasi momento per selezionare la navicella attualmente mostrata a schermo e tornare al Menu Principale.



Figura A.2: Come Appare Lo *Shop*

A.3 Gameplay e Controlli di Gioco

Una volta avviata la partita, l'obiettivo è sopravvivere al maggior numero di ondate nemiche possibili, accumulando punti e difendendo la propria salute.

- **Movimento:** La navicella si muove orizzontalmente. *Qui si possono scegliere i tasti A e D oppure anche le freccia Destra e quella Sinistra.*
- **Fuoco:** Premere *Barra Spaziatrice* per sparare i proiettili contro i nemici.
- **Bunker:** Il giocatore può sfruttare i bunker fluttuanti per ripararsi dai proiettili nemici. Attenzione: i bunker si deteriorano subendo danni.



Figura A.3: Come Appare la schermata di Gioco

A.4 Potenzamenti e Progressione

Al termine di un'ondata, il gioco entra in una fase di pausa in cui viene presentata una schermata di selezione dei Potenzamenti (Power-Up). Il giocatore può scegliere un singolo potenziamento (es. incremento danni, scudi, velocità) tramite i tasti freccia e confermare con Invio, per poi affrontare l'ondata successiva (che includerà nemici più resistenti o Boss).

Quando la salute del giocatore scende a zero, la partita termina (Game Over). Tutti i punti raccolti durante la run vengono sommati al proprio saldo totale per essere spesi nello Shop. Il sistema salva automaticamente su disco i progressi e il record del **Miglior Punteggio (High Score)**, che sarà visibile al riavvio dell'applicazione.



Figura A.4: Come Appare la schermata per scegliere i *PowerUp*

Appendice B

Esercitazioni di laboratorio

B.1 Andrea Ricci (andrea.ricci60@studio.unibo.it)

- Laboratorio 7: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284844>
- Laboratorio 8: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286129>
- Laboratorio 9: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287003>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288500>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289513>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290631>

B.2 Alessandro Tomè (alessandro.tome@studio.unibo.it)

- Laboratorio 9: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287243>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288494>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289496>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290615>

B.3 Enes Sela (enes.sela@studio.unibo.it)

- Laboratorio 6: <https://virtuale.unibo.it/mod/forum/discuss.php?d=206731#p284133>
- Laboratorio 7: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p285013>
- Laboratorio 8: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286119>
- Laboratorio 9: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p286925>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288518>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289531>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290638>